COSTRA — Review 3 Presenter Guide

Target duration: 8–10 minutes plus questions.
Presentation: `presentation/COSTRA_Final_Project_Review3_Demonstration.pptx`
Student: Arsh Shukla · 24BDS0113 · VIT Vellore

Before the review

1. Open the extracted project folder in VS Code.
2. In the terminal, run `python3 --version` and confirm Python 3.9 or newer.
3. Run `python3 -m unittest -v`. The current checked suite has 52 tests and should end with `OK`.
4. Run `python3 main.py` to open the professor-selectable menu.
5. Run `python3 main.py --demo` once if you want the complete output ready for reference.

Professor-selectable demo menu

Run `python3 main.py`; choose the number matching the request. COSTRA returns to the menu after each option.

1. **Review 1 — design:** problem, objectives, scope and architecture.
2. **Review 2 — core implementation:** lexer, AST, semantic checks, relational IR and a sample query result.
3. **Review 3 — full demonstration:** full multi-join query, candidates, selected estimated plan, SQLite output, SQLite query plan and timings.
4. **Enter a custom SELECT:** type the complete SQL on one line at the `SQL>` prompt and finish with a semicolon.
5. **Cost calculation / best choice:** a worked index-vs-scan example, all six demo candidate costs, and the minimum-cost candidate(s).
6. **Review-wise progress:** summarizes Review 1 → Review 2 → Review 3.
7. **Semantic error demonstration:** unknown-column rejection before SQL execution.
8. Exit.

Direct commands, when you need one stage without showing the menu:

```
python3 main.py --review1-demo
python3 main.py --review2-demo
python3 main.py --progress-demo
python3 main.py --cost-demo
python3 main.py --demo
python3 main.py --error-demo
python3 main.py --query "SELECT name FROM Employee LIMIT 2;"
python3 -m unittest -v
```

Slide-by-slide speaking notes (32 slides)

1. **Cover:** "COSTRA stands for Cost-Based SQL Compiler & Query Optimizer. This review shows the integrated system."
2. **Overview:** "I'll follow one SQL query through the major language and optimization stages."
3. **Project progress:** "Review 1 defined the design, Review 2 demonstrated core modules, and the final phase integrates execution and validation."
4. **Review milestones:** Explain the specific deliverables of Review 1, Review 2 and Review 3; these are project milestones and menu walkthroughs, not separate compilers.
5. **Motivation and scope:** "SQL is declarative. COSTRA makes the processing steps and plan reasoning visible for learning."
6. **Worked query:** Point out the joins, filters, group by, HAVING, ORDER BY and LIMIT.
7. **Live input:** If asked to enter SQL, run `python3 main.py`, select option 4, and type a one-line SELECT.
8. **Compiler concepts:** "The project demonstrates lexer, parser/AST, semantic checks, relational IR and cost-based analysis. SQLite is the separate execution backend."
9. **Architecture:** Walk through the pipeline and distinguish COSTRA's analytical optimizer from SQLite execution.
10. **Lexer:** Tokens recognized include keywords, identifiers, operators, literals and punctuation; lexical errors stop the pipeline.
11. **Parser / AST:** The parser checks grammar and precedence, then creates a structured representation of the SQL.
12. **Semantic analysis:** Explain table, alias and column validation and why invalid references should not reach SQLite.
13. **Relational IR:** Explain logical operators such as scan, join, filter, aggregate, HAVING, sort, limit and project.
14. **Cardinality:** "Cardinality estimates the number of rows; selectivity estimates the fraction that passes a predicate. Values here are synthetic."
15. **Plan alternatives:** The model compares access paths and nested-loop, hash and merge join costs; join enumeration has a cap.
16. **Candidate costs:** Plans 1 and 2 tie at 115,222.63 model units; plans 3–6 are higher.
17. **Cost formulas:** Explain full scan, index scan, nested-loop, hash, merge, aggregation, distinct, sort and projection terms.
18. **Worked cost example:** Employee `salary > 60000`: 10,000 estimated input rows; 7,200 estimated outputs; full scan plus predicate = 12,000.00; index estimate = 4,346.58, so the index path has the lower estimate.
19. **Selected plan:** The optimizer chooses a candidate with minimum estimated total cost. Plans 1 and 2 tie; the first minimum is returned by the current implementation.
20. **Technical boundary:** "SQLite does not execute the chosen COSTRA physical tree. It independently plans the normalized SQL."

21. **Normalized SQL:** Show the validated AST rendered as normalized SQL sent to SQLite.
22. **Results:** The current seed fixture returns Finance, Engineering and HR; distinguish these three actual sample rows from synthetic estimated rows.
23. **SQLite query plan:** This is SQLite's own `EXPLAIN QUERY PLAN` output, separate from COSTRA's estimate.
24. **Timings:** Observed local stage timings in milliseconds; one run on a tiny in-memory fixture is evidence, not a general benchmark.
25. **Error handling:** Show an invalid column rejected at semantic analysis before execution.
26. **Tests:** "The checked regression suite passes 52 tests covering lexer, parser, semantics, optimizer and execution/CLI cases."
27. **Source modules:** Explain the division of responsibilities across files.
28. **Interactive menu:** Show that the professor can select Review 1, Review 2, Review 3, custom input, cost formula, review progress or error handling without restarting VS Code.
29. **Recommended demo flow:** Run menu option 6, then 1, 2, 3, 4 and 5 as useful; show option 7 for error handling and then the test command.
30. **Limitations:** Synthetic statistics, bounded join-order search, SQL subset, and no selected-plan injection into SQLite.
31. **Viva preparation:** Use the questions below; answer directly and do not claim behavior beyond the code.
32. **Closing:** "Thank you. I'm ready to explain any module, formula, test or limitation."

What to say about cost and plan selection

COSTRA estimates the work of candidate access paths and join algorithms using estimated rows. It compares the candidates and selects the lowest estimated total under its own cost model. Cost values are **relative model units**, not milliseconds. SQLite independently chooses the actual execution plan for the normalized SQL.

Key formulas implemented in `optimizer.py`:

- Full scan: `N`
- Predicate evaluation: `0.20 × input_rows`
- Index scan: `2 × log2(max(2, N)) + 0.60 × output_rows`
- Nested-loop join: `0.15 × L × max(1, R)`
- Hash join: `1.10 × (L + R) + 0.25 × min(L, R)`
- Merge join: `0.80 × (L + R) + 0.45 × L × log2(max(2, L)) + 0.45 × R × log2(max(2, R))`

Worked access-path calculation for Employee (`N=10,000`, estimated output `7,200`):

- Full scan + predicate = `10,000 + 0.20 × 10,000 = 12,000.00`
- Index scan = `2 × log2(10,000) + 0.60 × 7,200 = 4,346.58`

- Therefore the estimated index path is lower in this example.

For the multi-join demo, the six costs are 115,222.63, 115,222.63, 121,702.63, 121,702.63, 123,862.63 and 123,862.63 model units. Plans 1 and 2 tie at the minimum. The implementation returns the first minimum candidate in its stable candidate list.

Likely professor questions and short answers

Why is this a Compiler Design project? SQL is treated as a source language and passed through tokenization, parsing/AST, semantic validation and a relational IR before optimization analysis.

What is cardinality? The estimated row count for a relation or intermediate operator.

What is selectivity? The estimated fraction of rows that satisfy a predicate.

Why use an AST? It represents the structural meaning of the query so later stages do not need to interpret raw SQL text.

Why use a relational IR? It separates logical query operations from source syntax and makes the operator tree inspectable.

What does the cost 115,222.63 mean? It is a relative synthetic cost estimate, not wall-clock time.

Why choose the lowest-cost plan? Under the same model, it has the smallest estimated total work among the generated candidates; it is not a guarantee of the fastest real execution.

Why do Plans 1 and 2 tie? Their cost model totals are equal at the displayed precision. The current code selects the first candidate returned at the minimum.

Does SQLite execute the plan selected by COSTRA? No. SQLite plans and executes normalized SQL independently.

Why are estimated rows different from returned rows? Planning statistics describe a synthetic larger dataset; execution uses a tiny seeded fixture.

What is the main limitation? Synthetic statistics/costs, bounded join-order enumeration, a SQL subset, and no physical-plan injection into SQLite.

What happens for an unknown column? Semantic analysis rejects it and the program reports the failed stage without executing the SQL.

How do you demonstrate your own input? Run `python3 main.py`, select option 4, and enter one supported SELECT statement at `SQL>`.

Test and screenshot checklist

- Run `python3 -m unittest -v`; current checked output reports 52 tests and `OK`.
- Capture a screenshot of the interactive menu.
- Capture a custom SQL input/output.
- Capture the six plan costs and the lowest-cost decision.
- Capture the 3 returned demo rows and SQLite's separate plan.
- Capture the semantic-error demo and final test summary.